

1) Why is double-spending a problem only in digital P2P money and not in physical cash?

Double-spending is mainly a problem in **digital peer-to-peer money** because digital data can be copied easily. If money exists as digital information, a dishonest person may try to send the same digital value to two different people. In a P2P system with no central authority constantly checking balances, the system needs some special way to make sure the same money is not spent twice.

In **physical cash**, this problem is much smaller because a physical note cannot be in two places at once. If I hand you a Rs. 1000 note, I no longer physically possess that same note. So the act of transfer itself naturally prevents me from spending that exact same note again.

Also, in digital systems with a **central bank or payment company**, the central authority can keep one master record and stop duplicate spending. The problem becomes especially serious when money is both **digital** and **peer-to-peer**, because then there is no single central party automatically stopping duplicates. That is why blockchain had to solve this problem through shared transaction history and consensus.

2) Why does changing one transaction force recomputation of later blocks?

Each block contains its own data and also the **hash of the previous block**. That means the blocks are connected in a chain. If you change even one transaction in an earlier block, the contents of that block change, and as a result its hash also changes.

Now the next block still contains the **old previous hash**, which no longer matches the modified earlier block. So the connection breaks. To repair it, you would need to update the next block so that it stores the new correct previous hash. But once you change that next block, its own hash changes too.

That same problem then moves forward to the next block after that. So one changed transaction causes a domino effect: block after block must be recomputed. In Proof-of-Work systems, this becomes even harder because each recomputed block must again satisfy the mining difficulty requirement. That is why changing history is so difficult.

3) Why is a hash useful if it does not “store” the original data?

A hash is useful because its purpose is not to store the original data — its purpose is to act like a **fingerprint** of the data. A fingerprint does not contain the whole person, but it is still useful for identification. In the same way, a hash does not contain the full block contents, but it strongly represents those contents.

If the original data changes even slightly, the hash changes a lot. So the hash is extremely useful for **integrity checking**. It lets the system detect whether the data has been altered, without needing to compare every detail manually all the time.

Hashes are also useful because they are **fixed-length and efficient**. A whole block may contain many transactions, but its hash is short and compact. So blockchain can use hashes to link blocks, detect tampering, and verify consistency without storing or repeating all earlier data inside every new block.

4) Why is nonce needed if transactions are fixed?

In a practical blockchain, especially a Proof-of-Work blockchain, a block is not accepted unless its hash satisfies some rule, like starting with a required number of zeros. But the transactions in the block are usually fixed, because they are the real data the block is supposed to record. You cannot keep changing the transaction contents just to get a better-looking hash.

That is why the **nonce** is needed. The nonce is an extra value that miners can change freely without changing the actual meaning of the transactions. By changing the nonce again and again, miners produce different hash outputs for the same block contents.

So the nonce gives miners something adjustable. Without it, if the transactions and previous hash were fixed, the block would keep producing the same hash every time, and there would be no way to search for a valid hash that meets the difficulty rule.

5) Why is decentralization not the same as privacy?

Decentralization and privacy are two different ideas. **Decentralization** means that control is not held by one central authority. The system is maintained by multiple participants or nodes instead of one single party like a bank or company.

But **privacy** means hiding information or limiting who can see it. A system can be decentralized and still be very transparent. In many blockchains, transaction history is widely visible, which actually increases auditability but may reduce privacy.

So decentralization answers the question, “Who controls the system?” while privacy answers the question, “Who can see the information?” A blockchain may remove central control but still expose transaction activity publicly or semi-publicly. That is why the two should never be confused.

6) Why is blockchain not suitable for every application?

Blockchain is not suitable for every application because it adds complexity, cost, and coordination overhead. If one trusted organization can easily manage a normal database, then using blockchain may be unnecessary and inefficient.

For example, in a normal company payroll system or internal attendance system, one central authority usually already exists and is accepted. In that case, a traditional database is often faster, cheaper, and simpler than a blockchain.

Blockchain makes more sense when multiple parties need to share and agree on data **without wanting one party to have full control**. So blockchain is useful for special cases involving trust distribution, shared records, and consensus — not as a universal replacement for all databases.

7) Why is the ledger “not the database”?

The lecture says the ledger is “not the database” because in distributed ledger systems, the important thing is not just stored data sitting in one place. Instead, many nodes keep their own local records or journals. The real challenge is making those different journals align into one shared accepted view.

So the **ledger** is better understood as the **agreed shared state** that emerges when these different journals are synchronized. It is not just one master table stored on one machine.

In simple words, a database is often a thing you can point to, like one central stored record. But in DLT, the ledger is more like the common truth produced through synchronization and consensus among many participants. That is why the lecture says the ledger is not simply the database.

8) Why can journals differ in DLT?

Journals can differ in DLT because each node has its own local view of shared data, and in a distributed network, information does not reach everyone at exactly the same time. One node may hear about a transaction first, while another receives it later.

Journals may also differ because some nodes may be temporarily offline, slower, or still validating information. So temporary mismatch is natural in distributed systems.

The important point is that these journals are not supposed to stay different forever. The goal of DLT is to keep them synchronized enough that they converge to one shared accepted view. So difference at one moment is not failure — permanent inability to converge would be failure.

9) Why is consensus hard in a distributed environment?

Consensus is hard in a distributed environment because there is no single central authority telling everyone what the correct version of the data is. Instead, many separate nodes must coordinate and agree among themselves.

This becomes difficult because:

- nodes may receive information at different times,
- some nodes may fail or disconnect,

- some may be dishonest or malicious,
- and different nodes may temporarily have different local views.

Even with all these problems, the network still has to reach one shared accepted version of history. That is why distributed consensus is such a major challenge and why blockchain needs carefully designed consensus mechanisms like Proof of Work or Proof of Stake.

10) Why does valid blockchain require both valid blocks and valid transactions?

A blockchain must have **valid blocks** because the structure of the chain must be correct. That means blocks must be properly linked, the previous hash references must match, and in PoW systems the block hash must satisfy the required criteria.

But that alone is not enough. The blockchain must also contain **valid transactions**. If a block is perfectly mined and perfectly linked, but it contains fake or unauthorized transactions, then the blockchain is still not legitimate.

So both levels matter:

- **block validity** ensures the chain structure is correct,
- **transaction validity** ensures the actual recorded activity is correct.

A strong answer in one line is:

A blockchain is only trustworthy when both its structure and its contents are valid.